

CSCE 608 DATABASE SYSTEMS · MODULE 04 · LECTURE NOTES

Hash Indexes, LSM Trees, and Concurrent Access

The alternatives, and making any of them thread-safe.

MODULE OUTCOMES

Compare hash indexes with B+trees and say when each wins.

Explain extendible and linear hashing.

Explain the LSM tree and the read/write amplification trade.

Implement latch crabbing for concurrent B+tree access.

Distinguish a latch from a lock and explain why both exist.

1 Hash indexes

	Hash index	B+tree
Point lookup	O(1) — one page read in principle.	$O(\log_f N)$ — three levels, of which two are normally cached, so effectively one read .
Ordering	None . The hash deliberately destroys it.	Full .
Supports	Exact match only.	Range scans, ORDER BY, prefix matching, MIN and MAX — all for free from the ordering.
Growth	Requires rehashing, unless the scheme is dynamic (§1.1).	Grows gracefully by splitting.
Use when	Exact-match lookup on a high-cardinality key, where the key is large enough that B+tree fanout suffers.	Essentially everything else.

THE ASYMPTOTIC ADVANTAGE MOSTLY EVAPORATES IN PRACTICE

$O(1)$ against $O(\log N)$ looks decisive on paper. It is not, for a specific reason: **the B+tree's logarithm is three, and two of those three levels are permanently in the buffer pool** because they are only a few megabytes. So both structures do about one disk read.

And the B+tree does far more for that one read. Range scans, ordered output, prefix matching, and MIN/MAX all come free from the ordering that hashing deliberately destroys.

So hash indexes win in a narrow band: exact-match lookups on a high-cardinality key, particularly where the key is large enough that it would badly reduce B+tree fanout.

The practical evidence is telling: PostgreSQL's hash indexes were not crash-safe — not replicated, not recoverable — until version 10, released in 2017. Nobody had noticed, because almost nobody used them.

- **Static hashing** requires choosing the table size in advance, and growing means rehashing every entry at once. For a live database that is an unacceptable stall.
- **Extendible hashing** keeps a *directory* of pointers indexed by the top d bits of the hash. When a bucket overflows, it splits, and only the directory doubles — the other buckets are untouched and are simply pointed to twice. The cost of growth is paid on the directory, which is small.
- **Linear hashing** dispenses with the directory. When any bucket overflows, it splits *the next bucket in a fixed round robin* — which is usually not the one that overflowed. Overflow chains handle the interim. This is counterintuitive and is precisely what removes the need for a directory, because the split point is known rather than looked up.
- **Both schemes spread the cost of growth** across many operations rather than concentrating it in one stall, which is the property a live system requires.

2 LSM trees

B+TREES WRITE BADLY

Inserting a single small row into a B+tree **dirtyes an entire page**, which must eventually be written back to disk.

Inserts arriving in random key order touch random pages, so the resulting write pattern is random — the access pattern storage devices handle worst, and the one the whole of Module 02 was about avoiding.

And the write amplification is severe: writing a 16 KB page to record a 50-byte change is a factor of roughly 300. On SSDs this matters twice over, because flash has limited write endurance.

The LSM tree inverts the design. Buffer writes in memory, flush them periodically as immutable sorted files, and merge those files in the background. **Every write that reaches disk is a large sequential write of a whole file**, which is the pattern storage handles best.

```

WRITES --> MEMTABLE (in memory, sorted: skip list or B-tree)
          | flush when full, as an immutable sorted file (SST)
          v
          L0: [SST] [SST] [SST]          key ranges may overlap
          | compaction merges and sorts
          v
          L1: [ SST ][ SST ][ SST ]      disjoint key ranges
          |
          L2: ...                        each level ~10x the previous

READ: memtable, then ALL of L0, then one file per level below.
     A Bloom filter per file skips files that cannot hold the key.

```

Bloom filters are not an optimisation here. Without them, a point lookup for a key that does not exist would have to read a file from every level to establish its absence. A Bloom filter answers 'definitely not present' in a few bytes of memory, so the read touches only the files that might actually contain the key. **LSM reads are impractical without them**, which is why every implementation has them.

Amplification	Definition	B+tree	LSM tree
Read	Pages read per logical read.	Low — one path down the tree.	High — several files may need checking, mitigated by Bloom filters.
Write	Bytes written to disk per logical byte written.	High — a whole page per small change, randomly placed.	Lower, and sequential — though compaction rewrites data several times as it moves down the levels.
Space	Disk space used per byte of live data.	Low — about 1.4× from partially full pages.	High — superseded versions persist until compaction removes them.

THE RUM CONJECTURE

Athanassoulis et al. framed this as the **RUM conjecture**: for any access method you can optimise **Read** cost, **Update** cost, and **Memory** (space) — but only two of the three.

B+trees take read and space. LSM trees take update and (partly) read, giving up space. Neither is a more mature version of the other; they occupy different corners of a genuine trade-off.

So the choice follows the workload. Read-heavy and range-scan-heavy favours B+trees — which is why relational databases use them. Write-heavy with mostly point lookups favours LSM — which is why RocksDB, Cassandra, and LevelDB use them, and why they sit underneath so much of the key-value infrastructure.

Recognising a trade-off as structural rather than circumstantial is what stops the argument about which is 'better', and it is the useful content of this section.

3 Concurrent access

A LATCH IS NOT A LOCK

The terminology is genuinely confusing, and the literature is inconsistent. This course uses the database convention:

A latch protects a data structure — a page, a B+tree node, a hash bucket. It is held for microseconds, for the duration of a single physical operation. It is not logged, not tracked in any table, and deadlock is prevented by imposing an acquisition *order* rather than by detection. In operating systems this is called a lock or a mutex.

A lock protects data — a row, a range, a table. It is held for the duration of a **transaction**, which may be seconds. It is tracked in a lock table, it participates in deadlock detection, and it is the subject of Module 09.

Both exist and both are necessary, and they operate at completely different timescales for completely different purposes. Confusing them is the main obstacle to reading the literature.

```
SEARCH (read):
  latch root (shared)
  loop: latch child (shared); RELEASE parent; descend

INSERT / DELETE (write):
  latch root (exclusive)
  loop: latch child (exclusive)
        if child is SAFE: release ALL ancestor latches
        descend

SAFE = the operation cannot propagate past this node:
  insert -> node is not full      (no split will reach the parent)
  delete -> node is more than half full (no merge will)
```

The safety test is the whole idea. Once a node can absorb the change without splitting or merging, no ancestor can possibly be affected by it, so every ancestor latch can be released immediately — which lets other threads descend behind you. The name 'crabbing' comes from holding two levels at once and releasing the upper only after acquiring the lower, like a crab's gait.

OPTIMISTIC DESCENT IS WHAT PRODUCTION SYSTEMS ACTUALLY DO

Pessimistic crabbing takes **exclusive** latches from the root downward on every insert and delete. **The root is therefore exclusively latched by every writer**, however briefly, which serialises all writers at a single point. On a multi-core machine this is immediately the bottleneck.

Optimistic descent assumes the common case: take *shared* latches all the way down, on the assumption that no split will be needed, and take an exclusive latch only on the leaf.

If it turns out a split is required, release everything and restart the descent pessimistically. Splits are rare — a node splits once per roughly f insertions — so the restart is rare, and the overwhelming majority of writes never contend at the root at all.

This is the standard approach, and it is a good example of a general pattern: make the common case cheap and detect the rare case rather than defending against it everywhere.

- **B^{link}-trees** give each node a pointer to its right sibling. A reader that arrives at a node mid-split, and finds the key it wants is no longer there, follows the sibling pointer rather than blocking or restarting. This decouples readers from writers almost entirely, and it is still the basis of most production implementations — PostgreSQL's B-tree is a B^{link}-tree.
- **Optimistic lock coupling** uses a version counter per node: read the version, read the node, re-read the version, and restart if it changed. No latch is taken for reading at all.
- **Latch ordering prevents deadlock.** Always acquire top to bottom and left to right, without exception. Any code path that violates the order can deadlock, and the deadlock will appear under load, in production, rarely.
- **Use reader-writer latches** so that concurrent readers do not exclude one another — which is the whole point, since reads dominate.
- **Test with thread sanitisers and sustained stress, not by reasoning.** Concurrency bugs in an index corrupt data *silently*: the tree remains structurally plausible and returns wrong answers. Project 1's requirement of eight threads and ten million operations verified against a reference map is the minimum honest test.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
CMU 15-445 — Lectures 7 and 9, Hash Tables and Index Concurrency	Extendible and linear hashing, and latch crabbing with the safe-node condition.
O'Neil et al. — The Log-Structured Merge-Tree (1996, free)	The original LSM paper. Short, and the amplification argument of §2 is already in it.

Resource	What it covers for this module
Athanassoulis et al. — Designing Access Methods: The RUM Conjecture (free)	The framing in §2 that makes the trade-off explicit.
Lehman & Yao — Efficient Locking for Concurrent Operations on B-Trees (1981, free)	The B ^{link} -tree of §3. Still the basis of PostgreSQL's implementation.
RocksDB wiki	A production LSM engine, documented in detail by its maintainers. The compaction and tuning pages are the practical side of §2.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement extendible hashing with directory doubling. Verify that a bucket split touches only that bucket and the directory.
- 2 Implement linear hashing and compare the worst-case latency of a single insert against static hashing with rehashing.
- 3 Compare your hash index against your B+tree on point lookups and on range scans. Report both, and the key size at which hashing starts to win on point lookups.
- 4 Implement a minimal LSM tree: memtable, flush to sorted files, and level-based compaction.
- 5 Measure read, write, and space amplification for your LSM and for your B+tree on the same workload. Place both on the RUM triangle.
- 6 Add Bloom filters and measure the reduction in files read per lookup for keys that are absent.
- 7 Implement pessimistic latch crabbing. Measure throughput against thread count and show the root becoming the bottleneck.
- 8 Implement optimistic descent and repeat. Report the scaling improvement and the restart rate.
- 9 Run your concurrent B+tree under a thread sanitiser with eight threads and ten million mixed operations, verifying against a reference map.
- 10 Deliberately violate latch ordering in one code path and construct a workload that deadlocks.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compare hash indexes and B+trees on five axes, and say when hashing wins.
- 2 Why does the hash index's $O(1)$ advantage mostly evaporate in practice?

- 3 Explain extendible and linear hashing and what each avoids.
- 4 Give two reasons B+trees write badly.
- 5 Describe the LSM structure and explain why Bloom filters are essential rather than optional.
- 6 State the RUM conjecture and say which two each of B+tree and LSM choose.
- 7 Distinguish a latch from a lock on four axes.
- 8 What is latch crabbing, and what is the safe-node condition?
- 9 Why is pessimistic crabbing a bottleneck, and what does optimistic descent do instead?
- 10 What is a B^{link}-tree and what does the sibling pointer achieve?